

Does Rust Compile Volatile Correctly?

Caspar Popova

University of Maryland

John Bender

Sandia National Laboratories

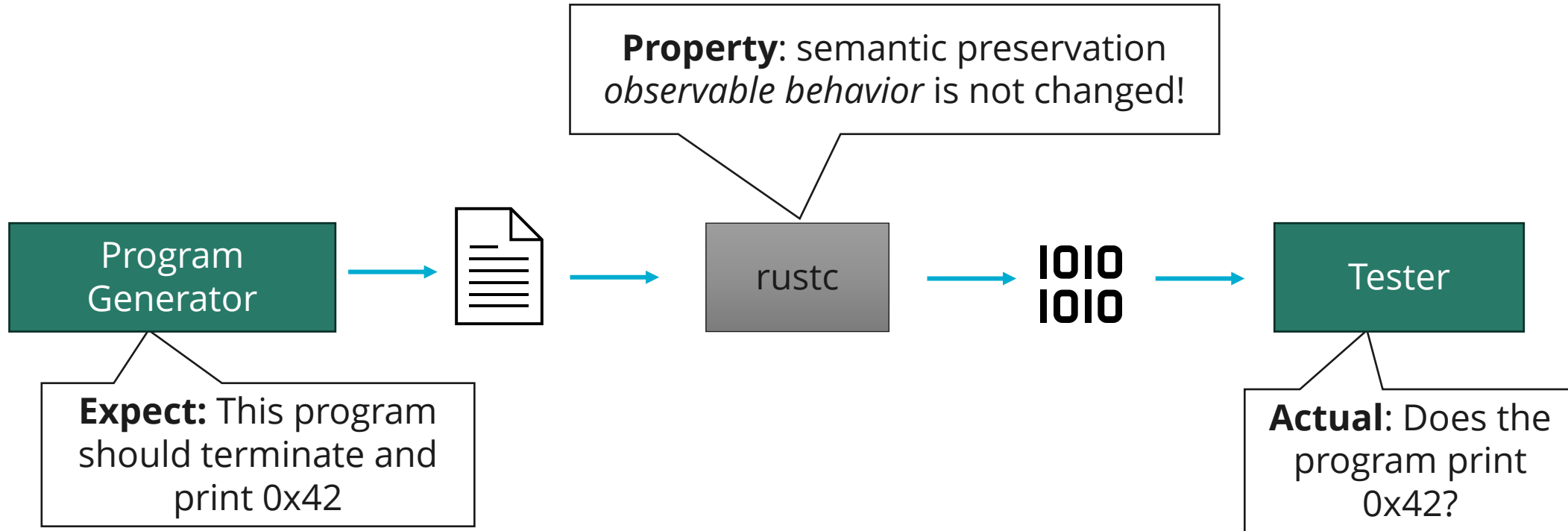
October 4, 2026 - SSNL '26

COMPILERS SHOULD BE CORRECT

- Verified compilers (eg CompCert) are great
- What about Rust?
 - Compilers have bugs
 - Testing for now!



COMPILER TESTING



DOES RUST COMPILE VOLATILE CORRECTLY?



Property:

- Compilers should not add or remove volatile accesses

Our work:

- Instrument binary to check volatile accesses
- Generate random Rust programs that use volatiles
- Lightweight simulation of embedded device

BACKGROUND: VOLATILE



Definition: A variable or memory location is *volatile* if its value can be changed by something other than the current thread of execution

- Example: MMIO in embedded devices

Property: Compilers may not optimize volatile by adding or removing accesses

```
fn rust_func() {  
    unsafe {  
        const UART_BASE: usize = 0x4000_8000;  
        let data_ptr = UART_BASE as *mut u32;  
        let v1 = core::ptr::read_volatile(data_ptr);  
        // ... hardware might write to data_ptr  
        let v2 = core::ptr::read_volatile(data_ptr); // can't be optimized out  
    }  
}
```

MOTIVATION: VOLATILE IS BUGGY

Example bugs from C/C++ compilers:

- Removing accesses
 - Caching value instead of re-reading
 - Removing struct members
- Adding accesses
 - Unconditional (rather than conditional) access

What about Rust?

Especially buggy when interacting with memory layouts!

structs,
unions, etc



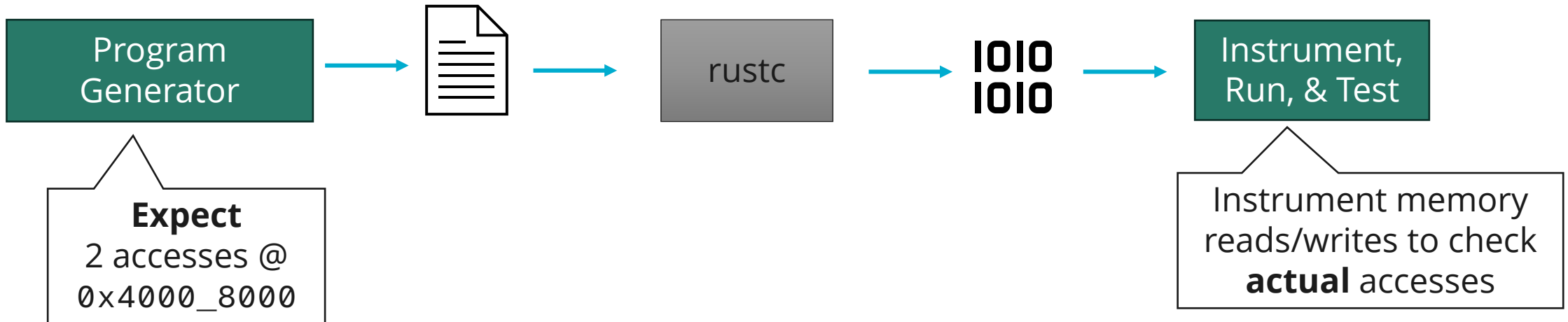
TESTING VOLATILE REQUIRES INSTRUMENTATION



Property: compiler does not alter volatile accesses

observable
behavior

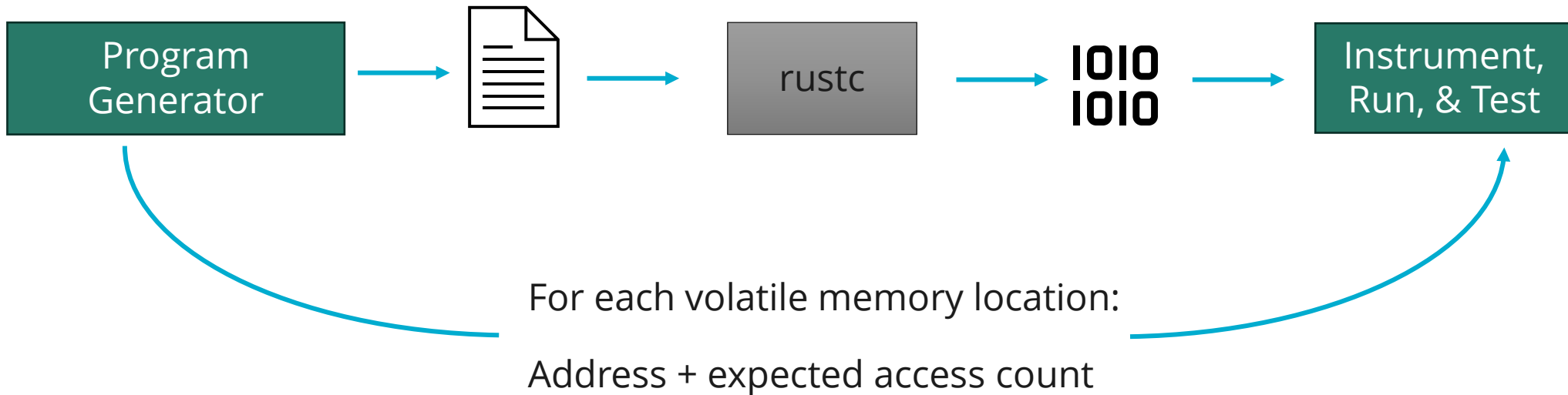
Check: instrument and check memory accesses



TESTING VOLATILE REQUIRES INSTRUMENTATION

Property: compiler does not alter volatile accesses

Check: instrument and check memory accesses

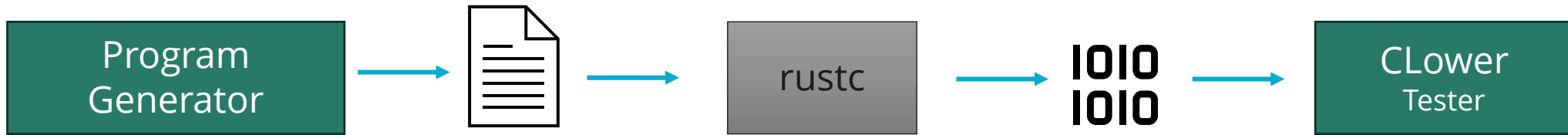


TESTING VOLATILE REQUIRES INSTRUMENTATION



Property: compiler does not alter volatile accesses

Check: instrument and check memory accesses



*Ground truth
testing w/
instrumentation*

CLower: Detecting Compiler Pessimization Bugs through Redundant Memory Accesses

JIANHAO XU, Southeast University, China

KUNBO ZHANG, State Key Laboratory for Novel Software Technology at Nanjing University, China

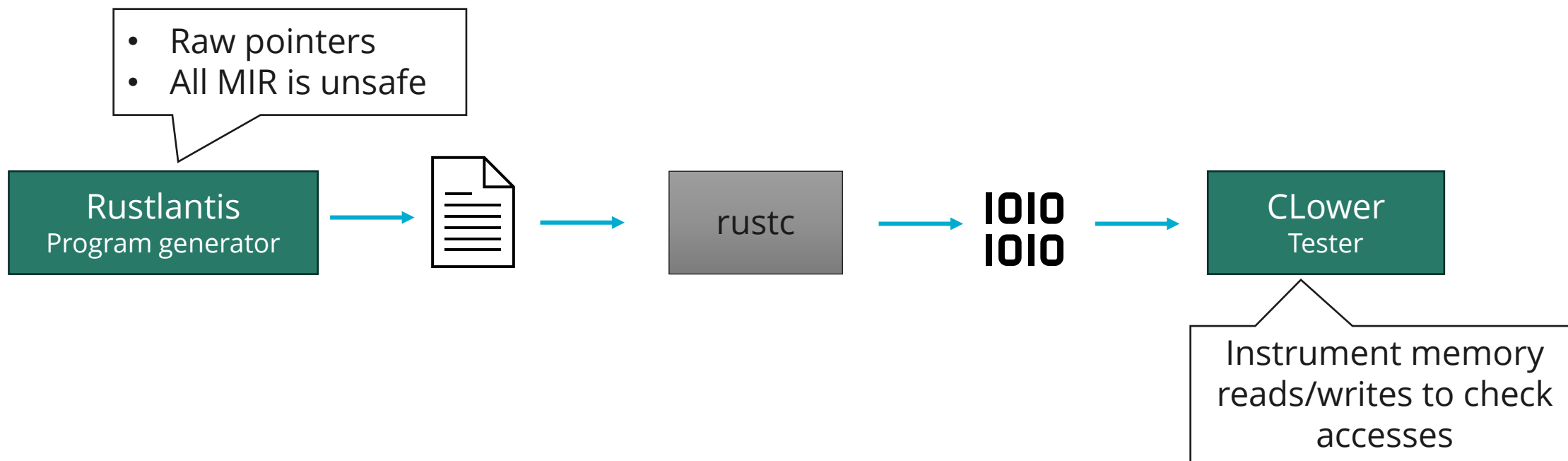
MATHIAS PAYER, EPFL, Switzerland

KANGJIE LU, University of Minnesota, USA

BING MAO, State Key Laboratory for Novel Software Technology at Nanjing University, China



GENERATING VOLATILE CODE



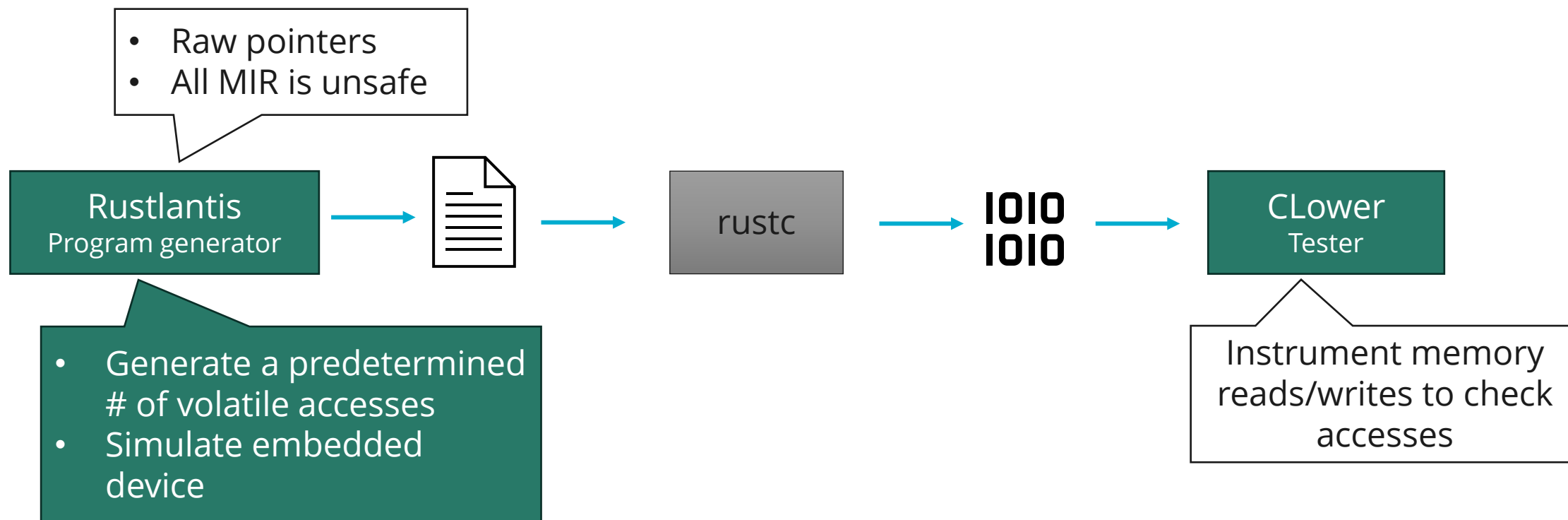
Rustlantis: Randomized Differential Testing of the Rust Compiler

QIAN WANG, ETH Zurich, Switzerland and Imperial College London, UK

RALF JUNG, ETH Zurich, Switzerland



GENERATING VOLATILE CODE



EXAMPLE: VOLATILE IN EMBEDDED DEVICES



Real: Device base address
Simulated: Malloc "embedded device memory"

```
fn rust_func() {  
    unsafe {  
        const UART_BASE: usize = 0x4000_8000;  
        const UART_DR: usize = UART_BASE + 0x00;  
        let data_ptr = UART_DR as *mut u32;  
        let v1 = core::ptr::read_volatile(data_ptr);  
        ...  
    }  
}
```

Real: Registers defined using scalar offsets
Simulated: Registers represented with memory layouts

```
#[repr(C)]  
struct RegisterBlock {  
    fld0: u32,  
    fld1: u32,  
    fld2: u64,  
}
```

GENERATING VOLATILE WITH SIMULATED EMBEDDED DEVICES



1. Memory address

```
fn rust_func(mut _1: u64) {  
    mir! {  
        let _2: RegisterBlock;  
        let _3: *mut RegisterBlock = _1 as *mut RegisterBlock;
```

Decoy struct

2. Generate a memory layout

```
#[repr(C)]  
struct  
RegisterBlock {  
    fld0: u32,  
    fld1: u32,  
    fld2: u64,  
}
```

3. Extend Rustlantis with *volatile* raw pointers

4. Update the memory model

- Avoid UB
- Generate accesses

```
// Decoy statement  
let _3 = &raw mut (_2);
```

GENERATING VOLATILE WITH SIMULATED EMBEDDED DEVICES



1. Device base address

```
fn rust_func(mut _1: u64) {  
    mir! {  
        let _2: RegisterBlock;  
        let _3: *mut RegisterBlock = _1 as *mut RegisterBlock;  
        let _4: *mut u32 = &raw mut ((*_3).fld0);  
        let _ = core::ptr::write_volatile(_4, 1171113538_u32)  
    }  
}
```

2. Generate an ADT

```
#[repr(C)]  
struct  
RegisterBlock {  
    fld0: u32,  
    fld1: u32,  
    fld2: u64,  
}
```

3. Extend Rustlantis with *volatile* raw pointers

4. Update the memory model

5. Generate accesses via projections

```
((*_3).fld0  
  
// Memory model: this  
location can be written
```

ONGOING AND FUTURE WORK

Ongoing

Adapting instrumentation to Rust

Memory layouts: unions

transmute
between variants

Future

More memory layouts

- Alignment annotations

Check access reordering

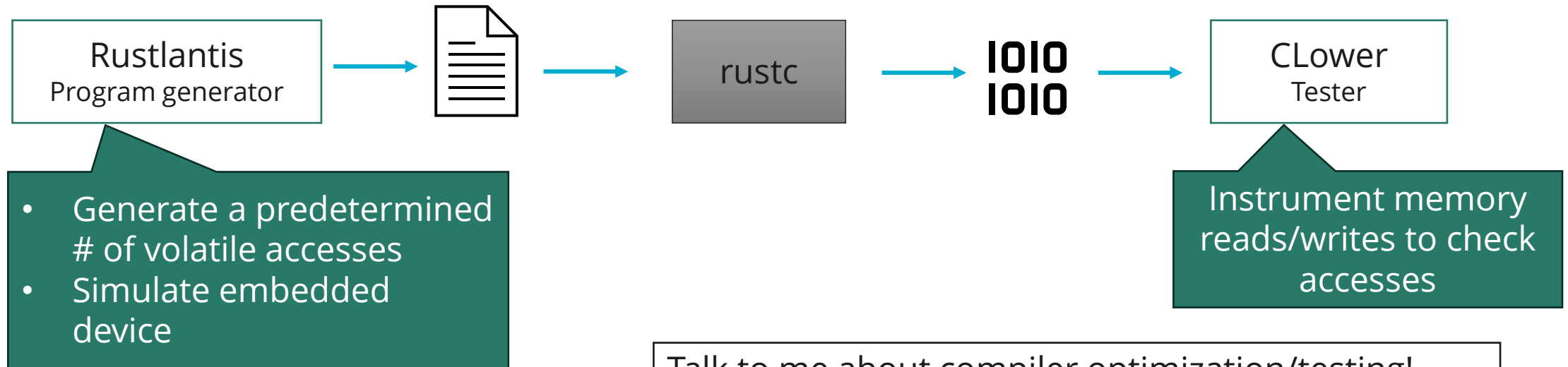


DOES RUST COMPILE VOLATILE CORRECTLY?



Our work:

- Instrument binary to check volatile accesses
- Generate random Rust programs that use volatiles
- Lightweight simulation of embedded device



Talk to me about compiler optimization/testing!

→ Caspar Popova (caspar@umd.edu)

REFERENCES



- Eric Eide and John Regehr. 2008. Volatiles are miscompiled, and what to do about it. In *Proceedings of the 8th ACM. International Conference on Embedded Software* (EMSOFT '08). Association for Computing Machinery, Atlanta, GA, USA, 255–264. isbn: 9781605584683. doi:10.1145/1450058.1450093.
- Qian Wang and Ralf Jung. 2024. Rustlantis: randomized differential testing of the rust compiler. *Proc. ACM Program. Lang.*, 8, OOPSLA2, Article 340, (Oct. 2024), 27 pages. doi:10.1145/3689780.
- Jianhao Xu, Kunbo Zhang, Mathias Payer, Kangjie Lu, and Bing Mao. 2026. Clower: detecting compiler pessimization bugs through redundant memory accesses. *Proc. ACM Program. Lang.*, 10, OOPSLA1, Article 142, (Apr. 2026), 27 pages. doi:10.1145/3798250.
- Robin Morisset, Pankaj Pawan, and Francesco Zappa Nardelli. 2013. Compiler testing via a theory of sound optimisations in the C11/C++11 memory model. *SIGPLAN Not.* 48, 6 (June 2013), 187–196. <https://doi.org/10.1145/2499370.2491967>
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in c compilers. *SIGPLAN Not.*, 46, 6, (June 2011), 283–294. doi:10.1145/1993316.1993532.